EECS 3311: Project Stage I

StudyBetter: An AI Study and Learning Agent

1. Project Overview

1.1 Project Description

What problem does your project solve?

University students taking several courses at once struggle to keep track of deadlines and grade weights scattered across syllabi and course websites. They often don't know which concepts they're weak in until an exam, and they have no clear way to decide what to study next, because existing tools only solve one piece: calendars track dates but don't teach, and general chatbots explain concepts but don't know a student's courses, deadlines, or progress. StudyBetter solves this by combining course organization, personalized learning, and progress-based study planning in one AI agent.

Who are the target users?

University and high school students are the target users for StudyBetter.

What can the agent do?

The agent can complete three main tasks: organizing, learning, and planning. It will read a syllabus and pull out deadlines, grade weights, and topics. It will help you learn concepts with explanations and active recall through quizzes, mock exams, and feedback. Also, it will track which concepts you have mastered and your progress in the course, and it will be able to create study plans tailored to priority tasks, weak concepts, and grades.

Why is an AI agent appropriate for this problem?

An AI agent is appropriate because it combines the abilities of different apps, like a calendar app and a chatbot, into one personalized study assistant that knows everything from your deadlines to your weakest topics. More importantly, many of its tasks require multiple steps and decisions rather than a single response. For example, to create a study plan, the agent must check upcoming deadlines, retrieve the student's weak concepts from past quiz results, consider how much each assessment is worth, and then decide what to prioritize. A regular app cannot reason about this data, and a regular chatbot does not have access to it or remember it between sessions. The agent uses tools, keeps memory of the student's progress, and makes decisions about what they should focus on next.

Which AI/LLM model(s) do you plan to use?

StudyBetter will use Google's Gemini model through the Gemini API as its main LLM. Gemini was chosen because its free tier is enough for development, it handles long inputs like the syllabus and lecture notes, and it supports tool calling, which the agent needs to retrieve course data. The system accesses the model through a common LLMClient interface, so the provider can be swapped without changing the rest of the system.

How will the AI model interact with the rest of the software system?

When the user makes a request through the GUI or CLI, it is sent to the AgentController, which passes it to the StudyAgent. The agent gathers the information it needs using tools, such as searching course materials, checking deadlines, or reading quiz results. It then sends this information to Gemini through the LLMClient interface. Gemini's response is turned into an object like a Quiz or StudyPlan, saved, and shown to the user. Simple features like grade calculation and deadline tracking do not use AI at all.

2. Feature Specifications

F01: Course and Material Management

Description: The user can add, edit, and delete courses and upload lecture notes or slides to each course. The agent summarizes each uploaded material into key points for quick review.
User Interaction: On the Courses page, the user clicks "Add Course" and fills in the details. Inside a course, they click "Upload Material," select a file, and view the generated summary beside it.
Input: Course code, name, term, instructor; material files (PDF, PowerPoint, text).
Output: Saved course; stored materials, each with a key-point summary.
AI Involvement: Hybrid. Course management and storage are deterministic; summarization uses the LLM.
Expected Workflow: 1. The user creates a course. 2. The user uploads a material. 3. The system extracts the text. 4. The agent asks the LLM for key points. 5. The summary is saved and displayed.
Error/Alternative Cases: Duplicate course code: show an error. Missing required fields: highlight them. Unsupported or unreadable file: reject with a message. LLM unavailable: save the file and allow summarizing later.

F02: Syllabus Import

Description: The user uploads the course syllabus, and the agent extracts assessments, due dates, grade weights, and the weekly topic schedule to set up the course for the other features.
User Interaction: On a course page, the user clicks "Import Syllabus," selects a file, reviews the extracted data in an editable table, and clicks "Confirm."
Input: Syllabus file (PDF or text); the selected course.
Output: Assessments (name, type, due date, weight) and weekly topics saved to the course.

AI Involvement: Hybrid. Text extraction is deterministic; the LLM identifies and structures the assessments, dates, weights, and topics.
Expected Workflow: 1. The user selects the file. 2. The system extracts the text. 3. The agent sends it to the LLM and requests structured output. 4. The response is converted into AssessmentItem and Topic objects. 5. The user reviews and edits. 6. Data is saved and appears in Deadline Tracking.
Error/Alternative Cases: Unreadable file: ask for another. Missing dates or weights: leave the fields blank and highlight them. Weights don't total 100%: warn before saving. LLM unavailable: allow manual entry.

F03: Deadline Tracking

Description: Shows all upcoming and overdue assessments across courses, alerts the user to deadlines that are due soon, and lets them mark items complete.
User Interaction: The Deadlines page shows a list or calendar view. The user can filter by course, sort by date or weight, and check off completed items.
Input: Assessments from F01/F02; filter and sort choices; completion status.
Output: Sorted deadline list or calendar; due-soon and overdue alerts.
AI Involvement: Deterministic.
Expected Workflow: 1. The user opens Deadlines. 2. The system loads all assessments. 3. Items are sorted and grouped (overdue, this week, later). 4. Alerts appear for items due within a set number of days. 5. The user marks items complete, and the status updates.
Error/Alternative Cases: No assessments yet: show an empty state that suggests importing a syllabus. Assessment missing a due date: list it under "No date." Past due and not complete: flag it as overdue.

F04: Grade Entry and Calculator

Description: The user enters marks for completed assessments, and the system calculates their current weighted grade and the score needed on remaining assessments to reach a target grade.
User Interaction: On the Grades tab of a course, the user enters a mark beside each assessment and optionally sets a target grade, for example 80%.
Input: Marks (score and out-of); assessment weights; optional target grade.
Output: Current weighted grade; required average on remaining work to reach the target.
AI Involvement: Deterministic.
Expected Workflow: 1. The user enters a mark. 2. The system validates it. 3. The current grade is recalculated from the weights. 4. If a target is set, the required score is calculated. 5. Results update on screen and in the Progress Dashboard.
Error/Alternative Cases: Mark greater than out-of, or negative: reject it. Weights missing: prompt the user to add them. Target unreachable: tell the user it's not achievable with the remaining weight.

F05: Concept Explanation

Description: The user asks about a concept, and the agent explains it using the student's own uploaded course materials and shows which lecture or file the explanation came from.

User Interaction: On the Learn page, the user selects a course, types a question (e.g. "What is the Factory Method pattern?"), and clicks "Explain." The answer appears with source references and an option to ask for a follow-up.
Input: Question text; selected course; the course's uploaded materials.
Output: A plain-language explanation with an example and the source material it was based on.
AI Involvement: AI-based. The agent searches the materials for relevant passages (a tool), then the LLM writes the explanation from them.
Expected Workflow: 1. The user submits a question. 2. The agent searches the course materials for relevant passages. 3. The agent sends the question and passages to the LLM. 4. The LLM generates an explanation. 5. The explanation and sources are displayed. 6. The concept is recorded so it can be included in future quizzes.
Error/Alternative Cases: No materials uploaded: answer from general knowledge and label it "not from your course materials." No relevant passages found: tell the user and offer a general explanation. Question too vague: the agent asks a clarifying question. LLM unavailable: show an error and let the user retry.

F06: Quiz Generation

Description: The agent creates a short practice quiz on topics the user chooses, using questions based on the course materials.
User Interaction: On the Practice page, the user selects a course, picks topics (or "Weak topics"), and chooses the number of questions and difficulty, then clicks "Generate Quiz." Questions appear one at a time.
Input: Course; selected topics; number of questions; difficulty; course materials.
Output: A quiz of multiple-choice and short-answer questions, each tagged with a concept and its correct answer.
AI Involvement: AI-based. The LLM generates questions and answers from the retrieved material.
Expected Workflow: 1. The user selects quiz settings. 2. The agent retrieves material for the chosen topics. 3. The agent asks the LLM for questions in a structured format. 4. The response is validated and converted into a Quiz object. 5. The quiz is displayed for the user to take. 6. Submitted answers go to F08 for grading.
Error/Alternative Cases: No materials for a topic: warn the user and generate from topic names only. LLM returns a badly formatted response: retry once, then show an error. Fewer valid questions than requested: show the ones generated and notify the user.

F07: Mock Exam Builder

Description: The agent builds a full-length timed practice exam that covers the course's topics, weighted like the real exam according to the syllabus.
User Interaction: On the Practice page, the user clicks "Mock Exam," selects the course and the exam it simulates, and sets the time limit. A timer runs while they answer, and they submit at the end.
Input: Course; target exam; topic weights from the syllabus; time limit; course materials.
Output: A timed mock exam; after submission, a score with a per-topic breakdown.
AI Involvement: Hybrid. Choosing how many questions come from each topic, and running the timer, are deterministic; question generation uses the LLM.

Expected Workflow: 1. The user selects the exam and time limit. 2. The system calculates the question distribution from the topic weights. 3. The agent generates questions per topic through the LLM. 4. The exam is assembled and the timer starts. 5. The user submits, or time runs out. 6. Answers are graded by F08, and a per-topic score report is shown.
Error/Alternative Cases: No syllabus topic weights: split the questions evenly across topics. Time runs out: auto-submit the answers so far. Generation fails partway: offer a shorter exam with the questions generated so far.

F08: Answer Grading and Feedback

Description: The system grades answers from quizzes and mock exams and explains each mistake. Users can also upload graded past tests from the course, and the agent analyzes where marks were lost, links each mistake to a concept, and explains it.
User Interaction: After submitting a quiz or mock exam, the user sees a results page with each question marked correct or incorrect, the correct answer, and feedback. To analyze a real test, the user clicks "Analyze Past Test" on the Practice page, uploads the file (PDF or photos), reviews the extracted questions and marks, and views the mistake report.
Input: Questions, the user's answers, and the correct answers or expected key points; or a graded past test file (PDF or image).
Output: A score, per-question correctness and feedback, and a list of mistakes grouped by concept.
AI Involvement: Hybrid. Multiple-choice grading and score totals are deterministic; the LLM grades short answers, reads uploaded tests (including scans, using Gemini's image input), tags questions with concepts, and explains mistakes.
Expected Workflow:
Practice tests: 1. The user submits answers to a quiz or mock exam. 2. Multiple choice is checked against the answer key. 3. Short answers are sent to the LLM for grading and feedback. 4. The total score is calculated and results are shown.
Past tests: 1. The user uploads the graded test. 2. The agent extracts the questions, answers, and marks. 3. The user reviews and corrects the extracted data. 4. The agent tags each question with a concept and explains the lost marks using the course materials.
Both: results are sent to F09 to update concept mastery.
Error/Alternative Cases: Blank answer: mark it incorrect and show the correct answer. LLM unavailable: mark short answers "pending" and allow self-grading, or save the past test to analyze later. Blurry or unreadable scan: ask for a clearer upload or manual entry. Marks not visible on the test: ask the user to enter them. User disputes a grade: let them flag it and exclude it from mastery scoring.

F09: Concept Mastery Tracking

Description: The system keeps a mastery score for each concept based on quiz and exam results, so the student and the agent know which topics are strong, weak, or untested.
User Interaction: On a course's Mastery tab, the user sees each concept with a mastery level (e.g. Weak, Developing, Strong) and can click a concept to see its quiz history or start a practice quiz on it.
Input: Graded results from F08, each tagged with a concept.
Output: An updated mastery score and level per concept; a list of weak concepts.

AI Involvement: Hybrid. Score updates use a deterministic formula; the LLM tags questions with concepts when they're generated.
Expected Workflow: 1. Graded results arrive from F08. 2. Each result is matched to its concept. 3. The mastery score is updated, with recent results weighted more heavily. 4. The level is recalculated. 5. The dashboard and study planner are notified of the change.
Error/Alternative Cases: Concept has no attempts: show it as "Untested." Question has no concept tag: count it toward the course score only. Mastery can be reset by the user if it's no longer accurate.

F10: Study Session Logging

Description: The user records study sessions (course, topic, and duration) so the system can track how much time they spend studying each course.
User Interaction: The user clicks "Start Session," picks a course and topic, and clicks "End Session" when finished. They can also add a past session manually.
Input: Course; topic; start and end time (or a manual duration); optional notes.
Output: A saved study session record.
AI Involvement: Deterministic.
Expected Workflow: 1. The user starts a session and selects a course and topic. 2. The system records the start time. 3. The user ends the session. 4. The duration is calculated and the session saved. 5. The dashboard and study planner are notified.
Error/Alternative Cases: Session left running for an unrealistic time (e.g. over 6 hours): ask the user to confirm or edit the duration. End time before start time when entered manually: reject it. App closed during a session: restore it on the next launch.

F11: Progress Dashboard

Description: Displays the student's overall progress with charts of study time, grades, concept mastery, and completed assessments for each course.
User Interaction: The user opens the Dashboard page and can switch between "All courses" and a single course, and change the time range (week, month, term).
Input: Data from F03, F04, F09, and F10; the selected course and time range.
Output: Charts and summary cards: study hours per course, grade trends, mastery by concept, and upcoming and completed deadlines.
AI Involvement: Deterministic.
Expected Workflow: 1. The user opens the dashboard. 2. The system loads the stored data. 3. Statistics are calculated for the selected range. 4. Charts and cards are displayed. 5. The dashboard updates automatically when new grades, sessions, or quiz results are saved.
Error/Alternative Cases: No data yet: show an empty state with suggested first steps. Partial data, such as no grades entered: show the available charts and hide the empty ones with a short message.

F12: Personalized Study Plan

Description: The agent builds a weekly study plan by combining upcoming deadlines, assessment weights, weak concepts, current grades, and recent study time, and updates the plan when this data changes.

User Interaction: On the Planner page, the user enters the hours they have available each day and clicks "Generate Plan." The plan appears as daily tasks, each with the reason it was chosen. The user can mark tasks done or ask to regenerate.
Input: Available study hours; deadlines and weights (F02/F03); grades (F04); concept mastery (F09); recent study sessions (F10).
Output: A weekly study plan with daily tasks (topic, activity, and time), each with a short reason (e.g. "Midterm in 5 days and Design Patterns is weak").
AI Involvement: AI-based (agent). The agent uses tools to gather data from multiple features, then the LLM reasons over it to prioritize and schedule tasks.
Expected Workflow: 1. The user enters available hours and requests a plan. 2. The agent calls its tools to collect deadlines, weights, grades, mastery, and study history. 3. The agent ranks priorities (urgent and high-weight items, weak concepts). 4. The agent sends this context to the LLM to build a schedule that fits the available hours. 5. The response is converted into a StudyPlan object and displayed. 6. When new results or deadlines come in, the user is prompted to update the plan.
Error/Alternative Cases: New user with no quiz data: base the plan on deadlines and weights only, and suggest a diagnostic quiz. More work than available hours: prioritize the most urgent and high-weight items and tell the user what was left out. No upcoming deadlines: focus the plan on weak concepts. LLM unavailable: generate a basic plan sorted by deadline and weight.

3. UML Class Diagram

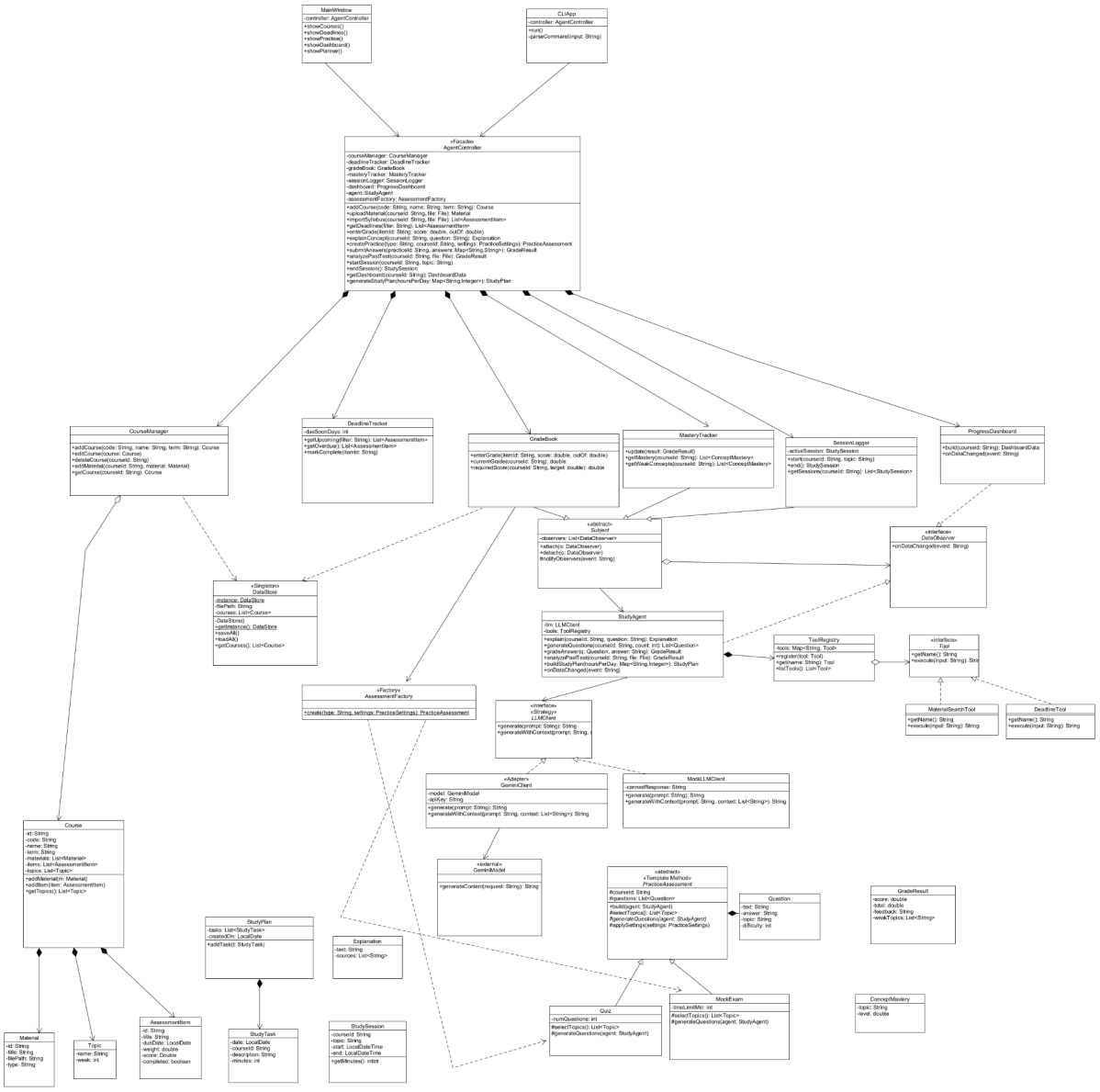


Figure 1: StudyBetter class diagram. The diagram is organized in layers: user interface (MainWindow, CLIApp), controller (AgentController), services (CourseManager, DeadlineTracker, GradeBook, MasteryTracker, SessionLogger, ProgressDashboard), persistence (DataStore), agent components (StudyAgent, ToolRegistry, Tool and its implementations, AssessmentFactory), the LLM layer (LLMClient, GeminiClient, MockLLMClient) and the domain model (Course, Material, AssessmentItem, Topic, PracticeAssessment, Quiz, MockExam, Question, StudyPlan, StudyTask, StudySession, GradeResult, ConceptMastery, Explanation).

4. Design Pattern Explanations

StudyBetter uses seven design patterns. Each one solves a specific problem in the design.

4.1 Facade: AgentController

Problem addressed: The GUI and the CLI both need to use many subsystems (courses, deadlines, grades, mastery, sessions, the dashboard and the AI agent). If each interface talked to every subsystem directly, both would be tightly coupled to all of them and the same logic would be written twice.

Participating classes and roles:

AgentController (Facade): provides one simple method per user action, such as explainConcept(), createPractice() and generateStudyPlan(), and forwards the work to the right subsystem.
MainWindow, CLIApp (Clients): call only AgentController.
CourseManager, DeadlineTracker, GradeBook, MasteryTracker, SessionLogger, ProgressDashboard, StudyAgent, AssessmentFactory (Subsystem classes): do the actual work.

Why it is appropriate: The project requires both a GUI and a CLI with the same functionality. A facade lets both interfaces share one entry point, so every feature is implemented once.

What would be harder without it: Without it, MainWindow and CLIApp would each need references to around eight classes, and any change to a subsystem would require changes in both interfaces.

4.2 Singleton: DataStore

Problem addressed: All services read and write the same saved data (courses, assessments, grades, sessions). If several DataStore objects existed, they could hold different copies of the data and overwrite each other's changes when saving.

Participating classes and roles:

DataStore (Singleton): has a private constructor, a static instance field and a static getInstance() method, so only one instance exists.
CourseManager, GradeBook (Clients): call DataStore.getInstance() to load and save data.

Why it is appropriate: There is exactly one data file for the user, so there should be exactly one object that controls it.

What would be harder without it: Without it, a reference to the store would have to be passed through every constructor, or separate instances could cause lost or inconsistent data.

4.3 Observer: Subject / DataObserver

Problem addressed: When a grade, quiz result or study session is saved, the Progress Dashboard has to update and the current study plan may become outdated. The services that change data should not need to know which screens or components depend on that data.

Participating classes and roles:

Subject (abstract Subject): keeps a list of observers and provides attach(), detach() and notifyObservers().
GradeBook, MasteryTracker, SessionLogger (Concrete Subjects): extend Subject and call notifyObservers() after their data changes.
DataObserver (Observer interface): declares onDataChanged(event).
ProgressDashboard, StudyAgent (Concrete Observers): ProgressDashboard refreshes its charts, and StudyAgent marks the current study plan as outdated.

Why it is appropriate: Several features depend on the same data changes (F09, F11, F12). Observer lets new listeners be added without editing the services.

What would be harder without it: Without it, GradeBook, MasteryTracker and SessionLogger would each have to call the dashboard and the agent directly, creating tight coupling and duplicated update code.

4.4 Factory Method: AssessmentFactory

Problem addressed: The user can request different types of practice (a quiz or a mock exam). The controller should not have to decide which class to create or how to set it up.

Participating classes and roles:

AssessmentFactory (Factory): its static create(type, settings) method returns the correct PracticeAssessment subclass.
PracticeAssessment (Product): the common abstract type returned to callers.
Quiz, MockExam (Concrete Products): the actual objects created.
AgentController (Client): asks the factory for a practice assessment by type.

Why it is appropriate: Object creation is kept in one place, and the rest of the system only works with the PracticeAssessment type.

What would be harder without it: Without it, AgentController would contain if/else logic for every practice type, and adding a new type (for example flashcards) would require editing the controller.

4.5 Strategy: LLMClient

Problem addressed: The agent needs an AI model, but the model provider may change, and testing should not depend on a live API with rate limits and costs.

Participating classes and roles:

LLMClient (Strategy interface): declares generate() and generateWithContext().
GeminiClient (Concrete Strategy): uses the real Gemini model.
MockLLMClient (Concrete Strategy): returns fixed responses for testing and offline use.

StudyAgent (Context): holds an LLMClient field and calls it without knowing which implementation it is.

Why it is appropriate: Choosing an AI model is a behaviour that should be swappable at runtime or in tests.

What would be harder without it: Without it, StudyAgent would be written directly against the Gemini SDK, making it impossible to test without network access and hard to switch to another model.

4.6 Adapter: GeminiClient

Problem addressed: Google's Gemini SDK has its own classes and method names (GeminiModel.generateContent()) that do not match the LLMClient interface the rest of the system expects.

Participating classes and roles:

LLMClient (Target): the interface StudyAgent uses.
GeminiModel (Adaptee): the external SDK class.
GeminiClient (Adapter): implements LLMClient and converts each call into a GeminiModel request, then converts the response back into a String.
StudyAgent (Client).

Why it is appropriate: Third-party code cannot be changed, so an adapter is needed to fit it into the existing interface.

What would be harder without it: Without it, SDK-specific code would spread throughout the agent, and an SDK update could break many classes instead of one.

4.7 Template Method: PracticeAssessment

Problem addressed: Quizzes and mock exams are built using the same overall steps (apply settings, choose topics, generate questions), but they choose topics and questions differently.

Participating classes and roles:

PracticeAssessment (Abstract Class): build() is the template method that runs the steps in a fixed order. selectTopics() and generateQuestions() are abstract steps, and applySettings() is a shared step.
Quiz (Concrete Class): selects the topics chosen by the user (or the weak topics) and generates a fixed number of questions.
MockExam (Concrete Class): selects topics weighted by the syllabus and generates a full-length timed exam.

Why it is appropriate: The algorithm structure is shared while specific steps vary, which is what Template Method is designed for.

What would be harder without it: Without it, the shared building steps would be duplicated in both classes, and the steps could be run in different orders by mistake.

5. Use-Case Diagram



Figure 2: StudyBetter use-case diagram.

Actors:

Student: the primary user, who uses all features through the GUI or CLI.
Gemini API: an external AI service used by the AI-based and hybrid use cases (UC01, UC02, UC05, UC06, UC07, UC08, UC12).

6. Use-Case Descriptions

UC01: Manage Courses and Materials

Actor(s): Student, Gemini API
Goal: Add, edit or delete a course and upload study materials with an AI summary.

Preconditions: The application is running.
Trigger: The student clicks "Add Course" or "Upload Material".
Main success scenario:
1. Student enters the course code, name, term and instructor.
2. System validates and saves the course.
3. Student selects a course and uploads a file.
4. System extracts the text and the agent requests a key-point summary from the LLM.
5. System saves the material with its summary and displays it.
Alternative / exception flows:
Duplicate course code: system shows an error.
Missing fields: system highlights them.
Unsupported file: system rejects it with a message.
LLM unavailable: the file is saved and can be summarized later.
Postconditions: The course and material are saved.
Related feature(s): F01

UC02: Import Syllabus

Actor(s): Student, Gemini API
Goal: Automatically set up a course's assessments, weights and topics from its syllabus.
Preconditions: The course exists.
Trigger: The student clicks "Import Syllabus".
Main success scenario:
1. Student selects the syllabus file.
2. System extracts the text.
3. Agent sends the text to the LLM and requests structured output.
4. System shows the extracted assessments, dates, weights and topics in an editable table.
5. Student reviews, edits and confirms.
6. System saves the data to the course.
Alternative / exception flows:
Unreadable file: system asks for another file.
Missing dates or weights: fields are left blank and highlighted.
Weights do not total 100%: system warns before saving.
LLM unavailable: student enters the data manually.
Postconditions: Assessments and topics are saved and appear in deadline tracking.
Related feature(s): F02

UC03: View and Manage Deadlines

Actor(s): Student
Goal: See upcoming and overdue assessments and mark them complete.
Preconditions: At least one assessment exists.
Trigger: The student opens the Deadlines page.
Main success scenario:
1. System loads all assessments.
2. System sorts and groups them into overdue, this week and later.
3. System shows alerts for items due soon.
4. Student filters or sorts the list.

5. Student marks an item complete and the system saves it.
Alternative / exception flows:
No assessments: system shows an empty state suggesting a syllabus import.
Missing due date: item is listed under "No date".
Postconditions: Completion status is saved.
Related feature(s): F03

UC04: Enter Grades

Actor(s): Student
Goal: Record marks and see the current grade and the score needed to reach a target.
Preconditions: The course has assessments with weights.
Trigger: The student opens the Grades tab.
Main success scenario:
1. Student enters a mark for an assessment.
2. System validates the mark.
3. System recalculates the current weighted grade.
4. If a target is set, system calculates the required score on the remaining work.
5. System displays the results and notifies the dashboard.
Alternative / exception flows:
Mark is negative or greater than out-of: system rejects it.
Target is unreachable: system informs the student.
Postconditions: The grade is saved and the dashboard is updated.
Related feature(s): F04

UC05: Ask for Concept Explanation

Actor(s): Student, Gemini API
Goal: Understand a concept using the student's own course materials.
Precondition: The course must exist.
Trigger: The student types a question and presses “Explain”
Main success scenario:
1. Student selects a course and submits a question.
2. Agent uses the material search tool to find relevant passages.
3. Agent sends the question and passages to the LLM.
4. LLM returns an explanation.
5. System displays the explanation with its sources.
Alternative / exception flows:
No materials uploaded: answer is given from general knowledge and labelled as such.
Question too vague: agent asks a clarifying question.
LLM unavailable: system shows an error and allows a retry.
Postconditions: The explanation is displayed and the concept is recorded for future practice.
Related feature(s): F05

UC06: Take a Practice Quiz

Actor(s): Student, Gemini API
Goal: Practise selected or weak topics with a generated quiz.

Preconditions: The course has topics.
Trigger: The student clicks "Generate Quiz".
Main success scenario:
1. Student chooses topics, number of questions and difficulty.
2. System creates a Quiz through the factory.
3. Agent retrieves material and asks the LLM for questions.
4. System validates the questions and displays the quiz.
5. Student answers and submits (continues in UC08).
Alternative / exception flows:
No material for a topic: questions are generated from topic names only.
Badly formatted LLM response: system retries once, then shows an error.
Postconditions: A quiz is generated and taken.
Related feature(s): F06

UC07: Take a Mock Exam

Actor(s): Student, Gemini API
Goal: Practise under exam conditions with a timed, syllabus-weighted exam.
Preconditions: The course has topics.
Trigger: The student clicks "Mock Exam".
Main success scenario:
1. Student selects the exam to simulate and the time limit.
2. System creates a MockExam through the factory.
3. System calculates how many questions come from each topic.
4. Agent generates the questions through the LLM.
5. System starts the timer and the student answers.
6. Student submits (continues in UC08).
Alternative / exception flows:
No topic weights: questions are split evenly.
Time runs out: answers are submitted automatically.
Generation fails partway: system offers a shorter exam.
Postconditions: A mock exam is completed and submitted.
Related feature(s): F07

UC08: Get Feedback on Answers

Actor(s): Student, Gemini API
Goal: Receive a score and an explanation for each mistake, for practice tests or uploaded past tests.
Preconditions: A quiz or mock exam was submitted, or the student has a graded past test file.
Trigger: The student submits answers or clicks "Analyze Past Test".
Main success scenario:
1. System checks multiple-choice answers against the answer key.
2. Agent sends short answers to the LLM for grading and feedback.
3. System calculates the total score.
4. System shows each question's result and feedback.
5. System sends the results to mastery tracking (UC09).

Alternative / exception flows:
Past test upload: agent extracts questions and marks, the student confirms them, and the agent explains lost marks.
LLM unavailable: short answers are marked "pending".
Unreadable scan: system asks for a clearer file or manual entry.
Postconditions: A GradeResult is saved and mastery is updated.
Related feature(s): F08

UC09: View Concept Mastery

Actor(s): Student
Goal: See which concepts are strong, developing, weak or untested.
Preconditions: At least one graded result exists.
Trigger: The student opens the Mastery tab.
Main success scenario:
1. System loads the mastery score for each concept.
2. System shows each concept with its level.
3. Student selects a concept to see its history or start a quiz on it.
Alternative / exception flows:
Concept has no attempts: shown as "Untested".
Student resets a concept's mastery.
Postconditions: The student knows which concepts to focus on.
Related feature(s): F09

UC10: Log Study Session

Actor(s): Student
Goal: Record time spent studying each course and topic.
Preconditions: The course exists.
Trigger:The student clicks “start” or “add past session”
Main success scenario:
1. Student selects a course and topic and starts the session.
2. System records the start time.
3. Student clicks "End Session".
4. System calculates the duration and saves the session.
5. System notifies the dashboard and planner.
Alternative / exception flows:
Unrealistic duration (over 6 hours): system asks the student to confirm.
End time before start time: system rejects it.
App closed during a session: the session is restored on the next launch.
Postconditions: The session is saved.
Related feature(s): F10

UC11: View Progress Dashboard

Actor(s): Student
Goal: See overall progress across courses.
Preconditions: The application is running.

Trigger: The student opens the Dashboard page.
Main success scenario:
1. System loads stored grades, sessions, mastery and deadlines.
2. System calculates statistics for the selected range.
3. System displays charts and summary cards.
Alternative / exception flows:
No data: empty state with suggested first steps.
Partial data: only available charts are shown.
Postconditions: Current progress is displayed, and it refreshes automatically when data changes.
Related feature(s): F11

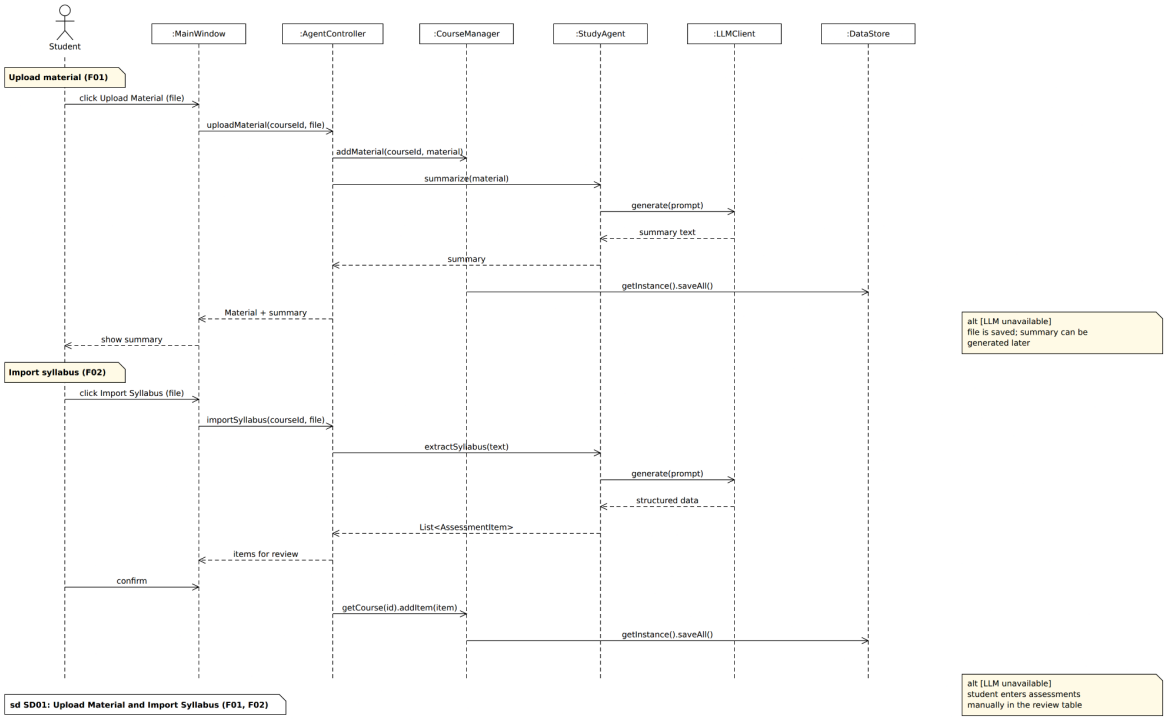
UC12: Generate Study Plan

Actor(s): Student, Gemini API
Goal: Get a weekly study plan based on deadlines, weights, weak concepts and grades.
Preconditions: At least one course exists.
Trigger: The student enters available hours and clicks "Generate Plan".
Main success scenario:
1. Student enters available study hours per day.
2. Agent uses its tools to gather deadlines, weights, grades, mastery and study history.
3. Agent ranks priorities.
4. Agent sends the context to the LLM to build a schedule.
5. System converts the response to a StudyPlan and displays it with a reason for each task.
Alternative / exception flows:
No quiz data: plan is based on deadlines and weights only.
Not enough hours: most urgent items are prioritized and the student is told what was left out.
LLM unavailable: a basic plan sorted by deadline and weight is generated.
Postconditions: A study plan is saved and displayed.
Related feature(s): F12

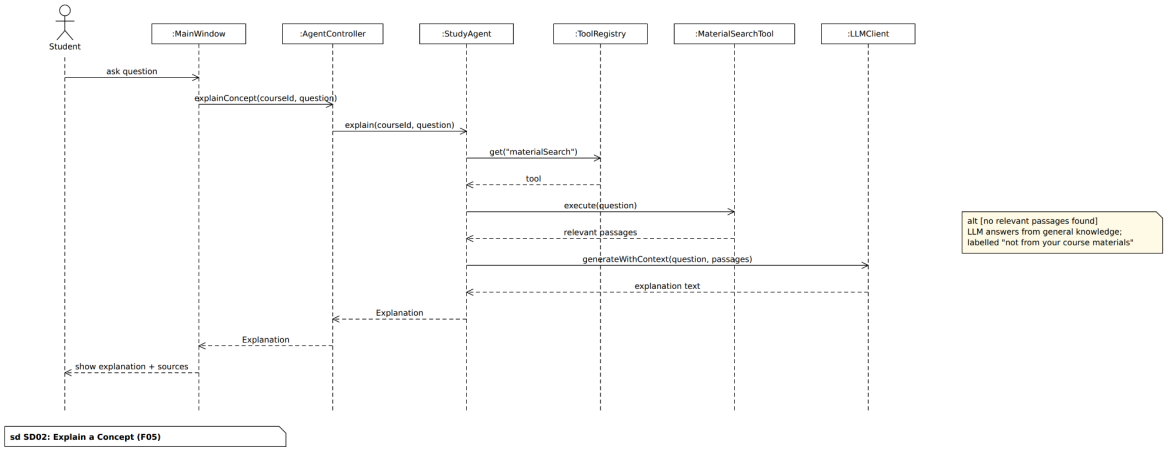
7. Sequence Diagrams

Six sequence diagrams cover all twelve features. Features with the same interaction structure share a diagram.

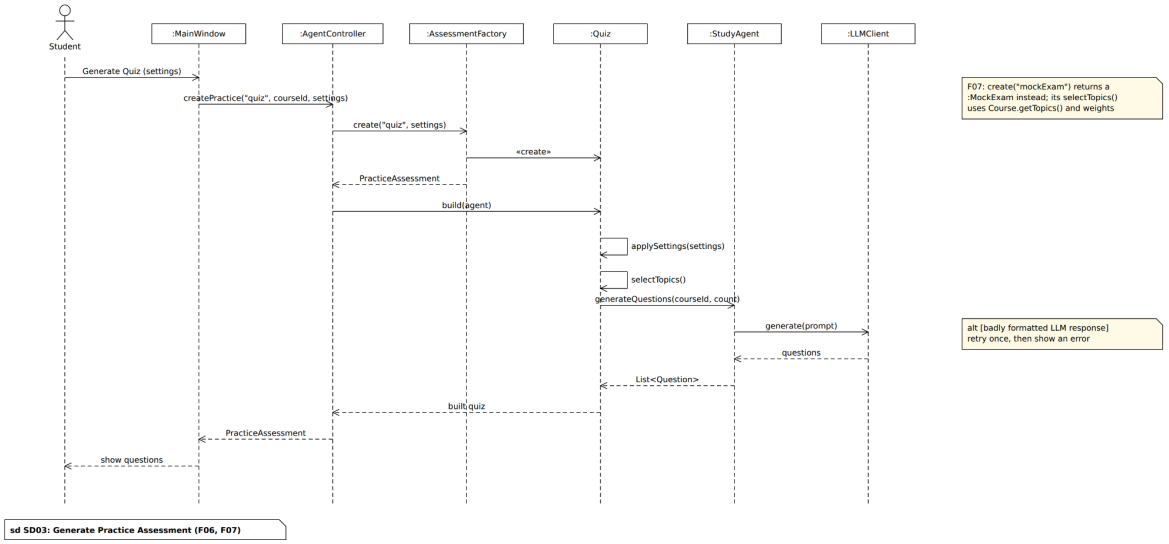
SD01: Upload Material and Import Syllabus
Features covered: F01, F02



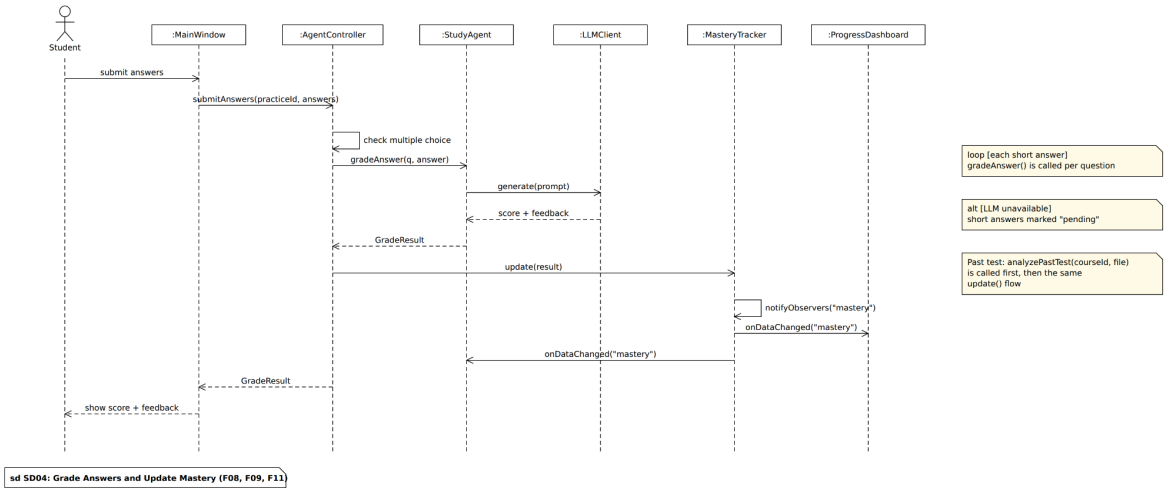
SD02: Explain a Concept (agent tool use)
Features covered: F05



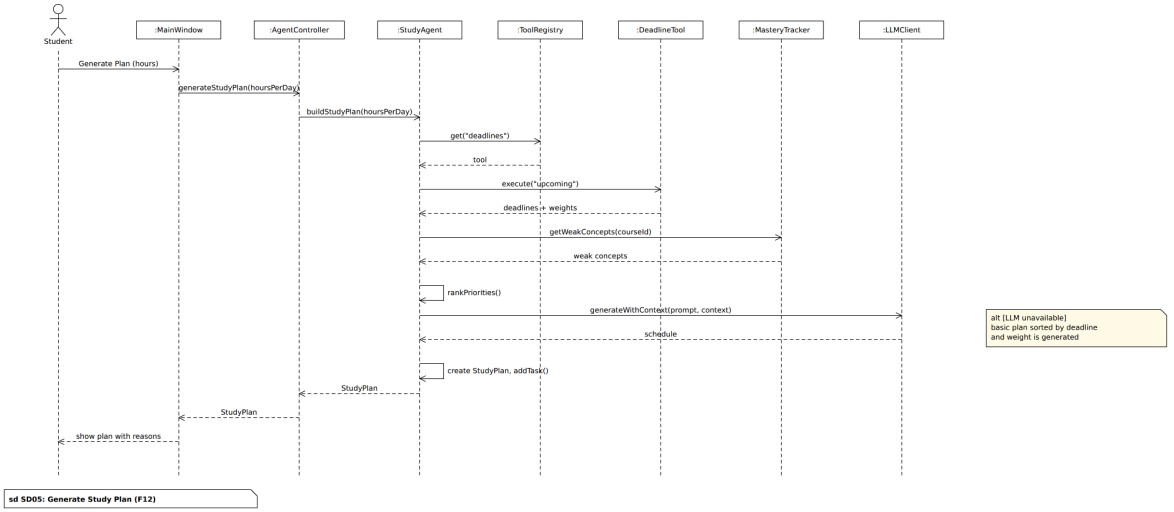
SD03: Generate Practice Assessment (Factory + Template Method)
Features covered: F06, F07



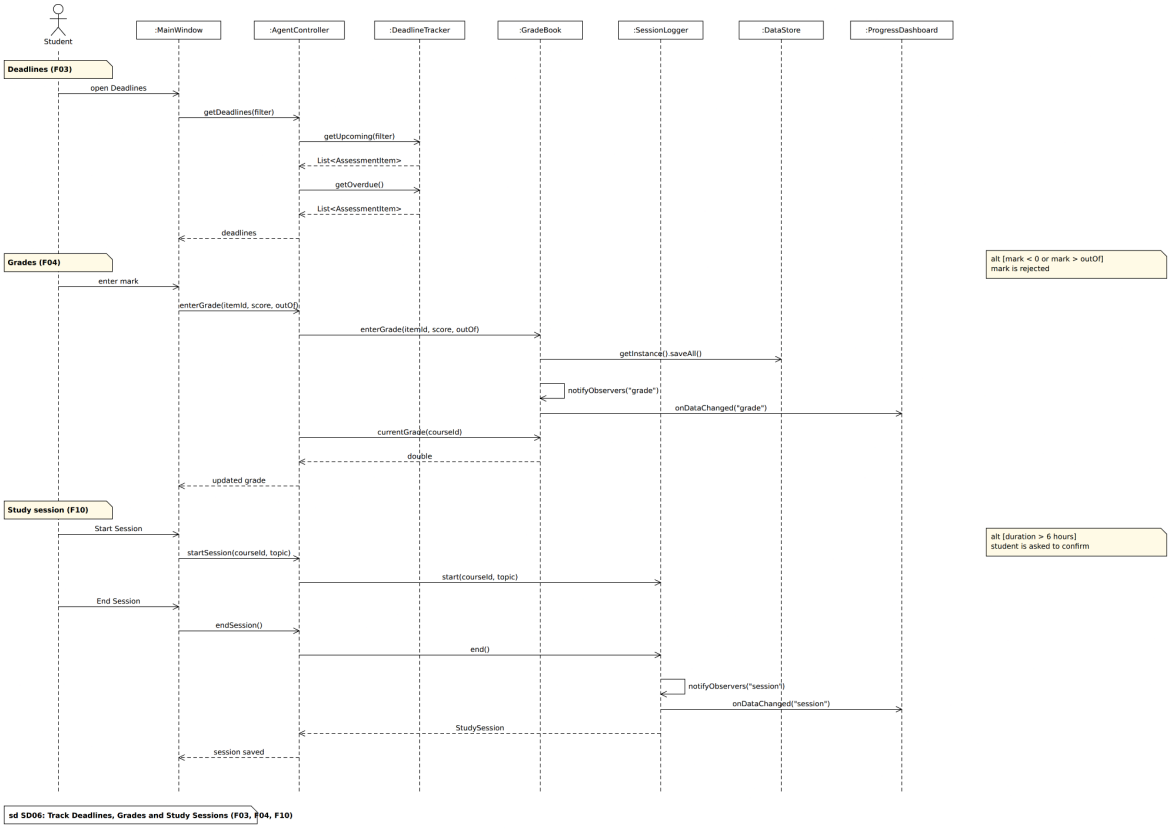
SD04: Grade Answers and Update Mastery (Observer)
Features covered: F08, F09, F11



SD05: Generate Study Plan (multi-step agent planning)
Features covered: F12



SD06: Track Deadlines, Grades and Study Sessions
Features covered: F03, F04, F10



8. Feature-to-Design Traceability Table

Feature	Description	Type	Use Case	Classes	Key Methods	SD	Pattern(s)
F01	Course and material management	Hybrid	UC01	MainWindow, AgentController, CourseManager, StudyAgent, LLMClient, DataStore, Course, Material	addCourse(), uploadMaterial(), addMaterial(), summarize(), generate(), saveAll()	SD01	Facade, Singleton, Strategy
F02	Syllabus import	Hybrid	UC02	MainWindow, AgentController, StudyAgent, LLMClient, CourseManager, Course, AssessmentItem, Topic	importSyllabus(), extractSyllabus(), generate(), addItem()	SD01	Facade, Strategy, Adapter
F03	Deadline tracking	Deterministic	UC03	MainWindow, AgentController, DeadlineTracker, AssessmentItem	getDeadlines(), getUpcoming(), getOverdue(), markComplete()	SD06	Facade
F04	Grade entry and calculator	Deterministic	UC04	MainWindow, AgentController, GradeBook, DataStore, ProgressDashboard	enterGrade(), currentGrade(), requiredScore(), notifyObservers()	SD06	Facade, Singleton, Observer
F05	Concept explanation	AI	UC05	MainWindow, AgentController, StudyAgent, ToolRegistry, MaterialSearchTool, LLMClient, Explanation	explainConcept(), explain(), get(), execute(), generateWithContext()	SD02	Facade, Strategy, Adapter
F06	Quiz generation	AI	UC06	AgentController, AssessmentFactory, Quiz, PracticeAssessment, StudyAgent, LLMClient, Question	createPractice(), create(), build(), selectTopics(), generateQuestions()	SD03	Factory Method, Template Method, Strategy
F07	Mock exam builder	Hybrid	UC07	AgentController, AssessmentFactory, MockExam, PracticeAssessment, StudyAgent, Course, Question	createPractice(), create(), build(), getTopics(), generateQuestions()	SD03	Factory Method, Template Method
F08	Answer grading and feedback	Hybrid	UC08	AgentController, StudyAgent, LLMClient, GradeResult, MasteryTracker	submitAnswers(), gradeAnswer(), analyzePastTest(), generate(), update()	SD04	Facade, Strategy
F09	Concept mastery tracking	Hybrid	UC09	MasteryTracker, ConceptMastery, Subject, DataObserver	update(), getMastery(), getWeakConcepts(), notifyObservers()	SD04	Observer
F10	Study session logging	Deterministic	UC10	MainWindow, AgentController, SessionLogger, StudySession	startSession(), endSession(), start(), end(), notifyObservers()	SD06	Facade, Observer
F11	Progress dashboard	Deterministic	UC11	MainWindow, AgentController, ProgressDashboard, DataObserver	getDashboard(), build(), onDataChanged()	SD04	Observer, Facade
F12	Personalized study plan	AI (agent)	UC12	MainWindow, AgentController, StudyAgent, ToolRegistry, DeadlineTool, MasteryTracker, LLMClient, StudyPlan, StudyTask	generateStudyPlan(), buildStudyPlan(), execute(), getWeakConcepts(), generateWithContext(), addTask()	SD05	Facade, Strategy, Observer

9. Feature Implementation Explanations

F01: Course and Material Management

Related use case: UC01
Related sequence diagram: SD01

Classes involved:
- MainWindow / CLIApp — collect course details and the uploaded file.
- AgentController — receives the request and coordinates it.
- CourseManager — creates, edits and deletes Course objects and attaches materials.
- StudyAgent — asks the LLM for a key-point summary.
- LLMClient — sends the prompt to Gemini.
- DataStore — saves the course data.
- Course, Material — store the course and its files.

Important methods:
- AgentController.addCourse()
- AgentController.uploadMaterial()
- CourseManager.addMaterial()
- StudyAgent.summarize()
- LLMClient.generate()
- DataStore.saveAll()

Execution: When the student adds a course, AgentController.addCourse() calls CourseManager, which creates a Course and saves it through DataStore.getInstance().saveAll(). When a file is uploaded, uploadMaterial() creates a Material, adds it to the course with addMaterial(), and asks StudyAgent.summarize() for key points. The agent calls LLMClient.generate(), and the summary is stored with the material and displayed.

F02: Syllabus Import

Related use case: UC02
Related sequence diagram: SD01

Classes involved:
- AgentController — receives the syllabus file.
- StudyAgent — builds the extraction prompt and parses the response.
- LLMClient / GeminiClient — return structured syllabus data.
- Course, AssessmentItem, Topic — store the extracted data.

Important methods:
- AgentController.importSyllabus()

- StudyAgent.extractSyllabus()
- LLMClient.generate()
- Course.addItem()

Execution: importSyllabus() passes the extracted text to StudyAgent.extractSyllabus(), which asks LLMClient.generate() for assessments, dates, weights and topics in a structured format. The response is converted into AssessmentItem and Topic objects and shown to the student for review. After confirmation, each item is added with Course.addItem() and saved.

F03: Deadline Tracking

Related use case: UC03
Related sequence diagram: SD06

Classes involved:
- MainWindow — displays the deadline list.
- AgentController — forwards the request.
- DeadlineTracker — filters and sorts assessment items.
- AssessmentItem — holds the due date and completion status.

Important methods:
- AgentController.getDeadlines()
- DeadlineTracker.getUpcoming()
- Deadline Tracker.getOverdue()
- DeadlineTracker.markComplete()

Execution: When the Deadlines page opens, getDeadlines() calls DeadlineTracker.getUpcoming() and getOverdue(), which compare each AssessmentItem's due date with today and the dueSoonDays setting. When the student checks off an item, markComplete() updates its status and the change is saved.

F04: Grade Entry and Calculator

Related use case: UC04
Related sequence diagram: SD06

Classes involved:
- AgentController — forwards the mark.
- GradeBook — validates marks and calculates grades; it is an Observer Subject.
- DataStore — saves the mark.
- ProgressDashboard — is notified of the change.

Important methods:
- AgentController.enterGrade()
- GradeBook.enterGrade()
- GradeBook.currentGrade()
- GradeBook.requiredScore()
- Subject.notifyObservers()

Execution: enterGrade() passes the mark to GradeBook, which validates it, stores it on the AssessmentItem and saves through DataStore. currentGrade() calculates the weighted average and requiredScore() calculates the score needed to reach the target. GradeBook then calls notifyObservers(), so ProgressDashboard refreshes automatically.

F05: Concept Explanation

Related use case: UC05
Related sequence diagram: SD02

Classes involved:
- AgentController — forwards the question.
- StudyAgent — plans the steps and builds the prompt.
- ToolRegistry — provides the available tools.
- MaterialSearchTool — searches course materials.
- LLMClient — generates the explanation.
- Explanation — holds the text and sources.

Important methods:
- AgentController.explainConcept()
- StudyAgent.explain()
- ToolRegistry.get()
- Tool.execute()
- LLMClient.generateWithContext()

Execution: explainConcept() calls StudyAgent.explain(). The agent gets MaterialSearchTool from ToolRegistry.get() and calls execute() to find relevant passages in the student's materials. It then calls LLMClient.generateWithContext() with the question and those passages. The result is returned as an Explanation with its sources and displayed.

F06: Quiz Generation

Related use case: UC06
Related sequence diagram: SD03

Classes involved:
- AgentController — receives the quiz settings.
- AssessmentFactory — creates a Quiz.
- Quiz / PracticeAssessment — run the template method to build the quiz.
- StudyAgent, LLMClient — generate the questions.
- Question — stores each question and answer.

Important methods:
- AgentController.createPractice()
- AssessmentFactory.create()
- PracticeAssessment.build()
- Quiz.selectTopics()

- Quiz.generateQuestions()

Execution: createPractice("quiz", settings) calls AssessmentFactory.create(), which returns a Quiz. The controller calls build(), the template method, which runs applySettings(), then Quiz.selectTopics() (chosen or weak topics), then Quiz.generateQuestions(), which uses StudyAgent.generateQuestions() and the LLM. The finished Quiz with its Question list is returned and displayed.

F07: Mock Exam Builder

Related use case: UC07
Related sequence diagram: SD03

Classes involved:
- AssessmentFactory — creates a MockExam.
- MockExam / PracticeAssessment — build a weighted, timed exam.
- Course — provides topics and weights.
- StudyAgent, LLMClient — generate questions per topic.

Important methods:
- AgentController.createPractice()
- AssessmentFactory.create()
- PracticeAssessment.build()
- Course.getTopics()
- MockExam.generateQuestions()

Execution: The flow is the same as F06, but the factory returns a MockExam. Its selectTopics() uses Course.getTopics() and the syllabus weights to decide how many questions come from each topic, and generateQuestions() requests them from the agent. The GUI starts a timer based on timeLimitMin.

F08: Answer Grading and Feedback

Related use case: UC08
Related sequence diagram: SD04

Classes involved:
- AgentController — receives the submitted answers or past test.
- StudyAgent — grades short answers and explains mistakes.
- LLMClient — produces feedback.
- GradeResult — holds the score, feedback and weak topics.
- MasteryTracker — receives the result.

Important methods:
- AgentController.submitAnswers()
- StudyAgent.gradeAnswer()
- StudyAgent.analyzePastTest()
- LLMClient.generate()

- MasteryTracker.update()

Execution: submitAnswers() checks multiple-choice answers against the stored answers, and sends each short answer to StudyAgent.gradeAnswer(), which uses the LLM for grading and feedback. The results are combined into a GradeResult. For past tests, analyzePastTest() extracts the questions and marks first. In both cases the GradeResult is passed to MasteryTracker.update().

F09: Concept Mastery Tracking

Related use case: UC09
Related sequence diagram: SD04

Classes involved:
- MasteryTracker — updates mastery scores; it is an Observer Subject.
- ConceptMastery — stores the score for one topic.
- DataObserver — is notified of changes.

Important methods:
- MasteryTracker.update()
- MasteryTracker.getMastery()
- MasteryTracker.getWeakConcepts()
- Subject.notifyObservers()

Execution: MasteryTracker.update() reads the weak topics and per-question results in the GradeResult and updates each ConceptMastery level, giving recent results more weight. It then calls notifyObservers("mastery"). getMastery() supplies the Mastery tab and getWeakConcepts() is used by quizzes and the study planner.

F10: Study Session Logging

Related use case: UC10
Related sequence diagram: SD06

Classes involved:
- AgentController — forwards start and end requests.
- SessionLogger — tracks the active session; it is an Observer Subject.
- StudySession — stores the course, topic and times.

Important methods:
- AgentController.startSession()
- SessionLogger.start()
- AgentController.endSession()
- SessionLogger.end()
- StudySession.getMinutes()

Execution: startSession() calls SessionLogger.start(), which creates a StudySession with the start time and stores it as activeSession. endSession() calls end(), which sets the end time,

calculates getMinutes(), saves the session and calls notifyObservers() so the dashboard and planner are updated.

F11: Progress Dashboard

Related use case: UC11
Related sequence diagram: SD04

Classes involved:
- MainWindow — displays charts.
- AgentController — forwards the request.
- ProgressDashboard — builds the statistics; it is a DataObserver.

Important methods:
- AgentController.getDashboard()
- ProgressDashboard.build()
- ProgressDashboard.onDataChanged()

Execution: getDashboard() calls ProgressDashboard.build(), which gathers data from GradeBook, MasteryTracker, SessionLogger and DeadlineTracker and returns DashboardData for the charts. Because ProgressDashboard is attached as an observer, onDataChanged() is called whenever grades, mastery or sessions change, so the dashboard refreshes without the user reloading it.

F12: Personalized Study Plan

Related use case: UC12
Related sequence diagram: SD05

Classes involved:
- AgentController — receives available hours.
- StudyAgent — runs the multi-step planning process.
- ToolRegistry, DeadlineTool — retrieve deadlines and weights.
- MasteryTracker — provides weak concepts.
- LLMClient — creates the schedule.
- StudyPlan, StudyTask — hold the plan.

Important methods:
- AgentController.generateStudyPlan()
- StudyAgent.buildStudyPlan()
- Tool.execute()
- MasteryTracker.getWeakConcepts()
- LLMClient.generateWithContext()
- StudyPlan.addTask()

Execution: generateStudyPlan() calls StudyAgent.buildStudyPlan(). The agent uses DeadlineTool.execute() to get upcoming deadlines and weights, asks MasteryTracker.getWeakConcepts() for weak topics, and collects grades and recent study

sessions. It ranks priorities and sends this context to LLMClient.generateWithContext() to build a schedule within the available hours. The response is converted into a StudyPlan with StudyTask objects using addTask(). Since StudyAgent is an observer, onDataChanged() marks the plan as outdated when new results arrive.